

Fragment API - float to Decimal Migration Report

Date: 2026-06-17

Author: OpenClaw (PBA Assistant)

Summary

Migration of monetary amount types from float to Decimal. DB models were already converted; this report covers the remaining service, sch

Files Changed

- src/fee.py - TON_FEE, after_fee(), after_ton_network_fee()
- src/config.py - API_PRICE_MARKUP, MIN_TON_DEPOSIT_AMOUNT, MIN_NON_SILENT_AMOUNT
- src/kit/schemas.py - Added DecimalFloat (Decimal serialized as float JSON)
- src/schemas.py - BaseBuyResponse.amount: float to DecimalFloat
- src/user/schemas.py - UserRead.balance: float to DecimalFloat
- src/fragment_transaction/schemas.py - All amount fields: float to DecimalFloat
- src/payment/endpoints.py - amount param: float to Decimal
- src/payment/service.py - amount params, to_amount() conversion
- src/fragment_transaction/service.py - float(to_amount()) to Decimal()
- src/exceptions.py - required_amount: float|None to Decimal|None
- src/backoffice/telegram_logs/fragment_transactions.py - float() to Decimal()
- src/integrations/fragment/types.py - BuyRequest.amount: float to Decimal

Tests Updated

- tests/fixtures/random_objects.py - Fixtures accept/return Decimal
- tests/payment/test_service.py - All comparisons use Decimal
- tests/payment/test_endpoints.py - Param amounts as strings
- tests/fragment_transaction/test_service.py - Decimal assertions

Not Changed (kept float)

- External API boundaries (ton_rate, thermos floor price)
- Non-monetary config (timeouts, retry sleeps)
- Time/rate tracking fields

Key Design

DecimalFloat: Pydantic Annotated type - Decimal at Python level, serialized as float in JSON. No API contract breakage.

Fee chain: Entire calculation chain uses Decimal (to_amount -> Decimal -> calc -> storage).

DB models: users.balance, payments.amount, fragment_transactions.amount were already on Decimal.